VGG16_Quantization_Aware_Training

December 12, 2025

[1]:
```
import argparse
import os
import time
import shutil

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.backends.cudnn as cudnn


import torchvision
import torchvision.transforms as transforms

from models import *


global best_prec
use_gpu = torch.cuda.is_available()
print('=> Building model...')



batch_size = 128
model_name = "VGG16_quant_8x8"
model = VGG16_quant_8x8()

print(model)

normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247, 0.243,␣
 ↪0.262])


train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
```

```
    download=True,
    transform=transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ]))
trainloader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size,␣
 ↪shuffle=True, num_workers=2)


test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))

testloader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size,␣
 ↪shuffle=False, num_workers=2)


print_freq = 100 # every 100 batches, accuracy printed. Here, each batch␣
 ↪includes "batch_size" data points
# CIFAR10 has 50,000 training data, and 10,000 validation data.

def train(trainloader, model, criterion, optimizer, epoch):
    batch_time = AverageMeter()
    data_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    model.train()

    end = time.time()
    for i, (input, target) in enumerate(trainloader):
        # measure data loading time
        data_time.update(time.time() - end)

        input, target = input.cuda(), target.cuda()

        # compute output
        output = model(input)
        loss = criterion(output, target)
```

```
        # measure accuracy and record loss
        prec = accuracy(output, target)[0]
        losses.update(loss.item(), input.size(0))
        top1.update(prec.item(), input.size(0))

        # compute gradient and do SGD step
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # measure elapsed time
        batch_time.update(time.time() - end)
        end = time.time()


        if i % print_freq == 0:
            print('Epoch: [{0}][{1}/{2}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Data {data_time.val:.3f} ({data_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   epoch, i, len(trainloader), batch_time=batch_time,
                   data_time=data_time, loss=losses, top1=top1))



def validate(val_loader, model, criterion ):
    batch_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    # switch to evaluate mode
    model.eval()

    end = time.time()
    with torch.no_grad():
        for i, (input, target) in enumerate(val_loader):

            input, target = input.cuda(), target.cuda()

            # compute output
            output = model(input)
            loss = criterion(output, target)

            # measure accuracy and record loss
            prec = accuracy(output, target)[0]
            losses.update(loss.item(), input.size(0))
```

```
            top1.update(prec.item(), input.size(0))

            # measure elapsed time
            batch_time.update(time.time() - end)
            end = time.time()

            if i % print_freq == 0:  # This line shows how frequently print out the status. e.g., i%5 => every 5 batch, prints out
                print('Test: [{0}/{1}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   i, len(val_loader), batch_time=batch_time, loss=losses,
                   top1=top1))

    print(' * Prec {top1.avg:.3f}% '.format(top1=top1))
    return top1.avg


def accuracy(output, target, topk=(1,)):
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk)
    batch_size = target.size(0)

    _, pred = output.topk(maxk, 1, True, True)
    pred = pred.t()
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].view(-1).float().sum(0)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res


class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
```

```
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count


def save_checkpoint(state, is_best, fdir):
    filepath = os.path.join(fdir, 'checkpoint.pth')
    torch.save(state, filepath)
    if is_best:
        shutil.copyfile(filepath, os.path.join(fdir, 'model_best.pth.tar'))


def adjust_learning_rate(optimizer, epoch):
    """For resnet, the lr starts from 0.1, and is divided by 10 at 80 and 120␣
 ↪epochs"""
    adjust_list = [150, 225]
    if epoch in adjust_list:
        for param_group in optimizer.param_groups:
            param_group['lr'] = param_group['lr'] * 0.1

#model = nn.DataParallel(model).cuda()
#all_params = checkpoint['state_dict']
#model.load_state_dict(all_params, strict=False)
#criterion = nn.CrossEntropyLoss().cuda()
#validate(testloader, model, criterion)
```

```
=> Building model…
VGG_quant(
  (features): Sequential(
    (0): QuantConv2d(
      3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): QuantConv2d(
      64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (5): ReLU(inplace=True)
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (7): QuantConv2d(
```

```
      64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (8): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (9): ReLU(inplace=True)
    (10): QuantConv2d(
      128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (11): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (12): ReLU(inplace=True)
    (13): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (14): QuantConv2d(
      128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (15): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (16): ReLU(inplace=True)
    (17): QuantConv2d(
      256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (18): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (19): ReLU(inplace=True)
    (20): QuantConv2d(
      256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (21): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (22): ReLU(inplace=True)
    (23): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (24): QuantConv2d(
      256, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (25): BatchNorm2d(8, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (26): ReLU(inplace=True)
    (27): QuantConv2d(
      8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
```

```
    )
    (28): ReLU(inplace=True)
    (29): QuantConv2d(
      8, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (30): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (31): ReLU(inplace=True)
    (32): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (33): QuantConv2d(
      256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (34): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (35): ReLU(inplace=True)
    (36): QuantConv2d(
      512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (37): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (38): ReLU(inplace=True)
    (39): QuantConv2d(
      512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (40): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (41): ReLU(inplace=True)
    (42): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (43): AvgPool2d(kernel_size=1, stride=1, padding=0)
  )
  (classifier): Linear(in_features=512, out_features=10, bias=True)
)
Files already downloaded and verified
Files already downloaded and verified
```

[2]:
```
model.features[27] ## Modified layer
```

[2]:
```
QuantConv2d(
  8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
```

[3]:
```
import os
os.getcwd()
```

[3]:
```
'/home/zix036/private/Project1.1_Vanilla'
```

[4]:
```
PATH = "/home/zix036/private/Project1.1_Vanilla/result/VGG_quant_8x8/model_best.
 ↪pth.tar"
checkpoint = torch.load(PATH)
model.load_state_dict(checkpoint['state_dict'])
device = torch.device("cuda")

model.cuda()
model.eval()

test_loss = 0
correct = 0

with torch.no_grad():
    for data, target in testloader:
        data, target = data.to(device), target.to(device) # loading to GPU
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()

test_loss /= len(testloader.dataset)

print('\nTest set: Accuracy: {}/{} ({:.0f}%)\n'.format(
        correct, len(testloader.dataset),
        100. * correct / len(testloader.dataset)))
```

```
Test set: Accuracy: 9008/10000 (90%)
```

[5]:
```
class SaveOutput:
    def __init__(self):
        self.outputs = []
    def __call__(self, module, module_in):
        self.outputs.append(module_in)
    def clear(self):
        self.outputs = []

########## Save inputs from selected layer ##########
save_output = SaveOutput()
i = 0
count = 0
for layer in model.modules():
```

```
    i = i+1
    if isinstance(layer, QuantConv2d):
        count += 1
        print(i,"-th layer prehooked \n", layer, count) #'count' here is just␣
 ↪for me to check the index of save_output.outputs[][0]
        layer.register_forward_pre_hook(save_output)
#####################################################

dataiter = iter(testloader)
images, labels = next(dataiter)
images = images.to(device)
out = model(images)
```

```
3 -th layer prehooked
 QuantConv2d(
  3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 1
7 -th layer prehooked
 QuantConv2d(
  64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 2
12 -th layer prehooked
 QuantConv2d(
  64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 3
16 -th layer prehooked
 QuantConv2d(
  128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 4
21 -th layer prehooked
 QuantConv2d(
  128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 5
25 -th layer prehooked
 QuantConv2d(
  256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 6
29 -th layer prehooked
 QuantConv2d(
  256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
```

```
) 7
34 -th layer prehooked
 QuantConv2d(
  256, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 8
38 -th layer prehooked
 QuantConv2d(
  8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 9
41 -th layer prehooked
 QuantConv2d(
  8, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 10
46 -th layer prehooked
 QuantConv2d(
  256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 11
50 -th layer prehooked
 QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 12
54 -th layer prehooked
 QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 13
```

[6]:
```
weight_q = model.features[27].weight_q
w_alpha = model.features[27].weight_quant.wgt_alpha
w_bit = 4

weight_int = weight_q / (w_alpha / (2**(w_bit-1)-1))
#print(weight_int)
```

[7]:
```
act = save_output.outputs[8][0]
act_alpha  = model.features[27].act_alpha
act_bit = 4
act_quant_fn = act_quantization(act_bit)

act_q = act_quant_fn(act, act_alpha)

act_int = act_q / (act_alpha / (2**act_bit-1))
```

```
#print(act_int )
```

[8]:
```
conv_int = torch.nn.Conv2d(in_channels = 64, out_channels=64, kernel_size = 3,␣
 ↪padding=1)
conv_int.weight = torch.nn.parameter.Parameter(weight_int)
relu = torch.nn.ReLU()
conv_int.bias = model.features[27].bias
output_int = conv_int(act_int)
output_recovered = output_int * (act_alpha / (2**act_bit-1)) * (w_alpha /␣
 ↪(2**(w_bit-1)-1))
output_recovered_relu = relu(output_recovered)
#print(output_recovered)
```

[9]:
```
difference = abs(save_output.outputs[9][0] - output_recovered_relu )
print(difference.mean())  ## It should be small, e.g.,2.3 in my trainned model
```

```
tensor(1.0516e-07, device='cuda:0', grad_fn=<MeanBackward0>)
```

[10]:
```
#print (relu(output_int))
```

[11]:
```
# act_int.size = torch.Size([128, 64, 32, 32])  <- batch_size, input_ch, ni, nj
a_int = act_int[0,:,:,:]  # pick only one input out of batch
# a_int.size() = [64, 32, 32]

# conv_int.weight.size() = torch.Size([64, 64, 3, 3])  <- output_ch, input_ch,␣
 ↪ki, kj
w_int = torch.reshape(weight_int, (weight_int.size(0), weight_int.size(1), -1))␣
 ↪ # merge ki, kj index to kij
# w_int.weight.size() = torch.Size([64, 64, 9])

padding = 1
stride = 1
array_size = 8 # row and column number

nig = range(a_int.size(1))  ## ni group
njg = range(a_int.size(2))  ## nj group

icg = range(int(w_int.size(1)))  ## input channel
ocg = range(int(w_int.size(0)))  ## output channel

ic_tileg = range(int(len(icg)/array_size))
oc_tileg = range(int(len(ocg)/array_size))

kijg = range(w_int.size(2))
ki_dim = int(math.sqrt(w_int.size(2)))  ## Kernel's 1 dim size

######## Padding before Convolution #######
```

```
a_pad = torch.zeros(len(icg), len(nig)+padding*2, len(nig)+padding*2).cuda()
# a_pad.size() = [64, 32+2pad, 32+2pad]
a_pad[ :, padding:padding+len(nig), padding:padding+len(njg)] = a_int.cuda()
a_pad = torch.reshape(a_pad, (a_pad.size(0), -1))
# a_pad.size() = [64, (32+2pad)*(32+2pad)]


a_tile = torch.zeros(len(ic_tileg), array_size,    a_pad.size(1)).cuda()
w_tile = torch.zeros(len(oc_tileg)*len(ic_tileg), array_size, array_size,␣
↪len(kijg)).cuda()

for ic_tile in ic_tileg:
    a_tile[ic_tile,:,:] = a_pad[ic_tile*array_size:(ic_tile+1)*array_size,:]

for ic_tile in ic_tileg:
    for oc_tile in oc_tileg:
        w_tile[oc_tile*len(oc_tileg) + ic_tile,:,:,:] =␣
↪w_int[oc_tile*array_size:(oc_tile+1)*array_size, ic_tile*array_size:
↪(ic_tile+1)*array_size, :]



##########################################

p_nijg = range(a_pad.size(1)) ## psum nij group

psum = torch.zeros(len(ic_tileg), len(oc_tileg), array_size, len(p_nijg),␣
↪len(kijg)).cuda()

for kij in kijg:
    for ic_tile in ic_tileg:       # Tiling into array_sizeXarray_size array
        for oc_tile in oc_tileg:   # Tiling into array_sizeXarray_size array   ␣
↪
            for nij in p_nijg:       # time domain, sequentially given input
                m = nn.Linear(array_size, array_size, bias=False)
                #m.weight = torch.nn.Parameter(w_int[oc_tile*array_size:
↪(oc_tile+1)*array_size, ic_tile*array_size:(ic_tile+1)*array_size, kij])
                m.weight = torch.nn.
↪Parameter(w_tile[len(oc_tileg)*oc_tile+ic_tile,:,:,kij])
                psum[ic_tile, oc_tile, :, nij, kij] = m(a_tile[ic_tile,:
↪,nij]).cuda()
```

[12]:
```
print(oc_tileg)
```

```
range(0, 1)
```

[13]:
```
import math

a_pad_ni_dim = int(math.sqrt(a_pad.size(1))) # 32

o_ni_dim = int((a_pad_ni_dim - (ki_dim- 1) - 1)/stride + 1)
o_nijg = range(o_ni_dim**2)

out = torch.zeros(len(ocg), len(o_nijg)).cuda()


### SFP accumulation ###
for o_nij in o_nijg:
    for kij in kijg:
        for ic_tile in ic_tileg:
            for oc_tile in oc_tileg:
                out[oc_tile*array_size:(oc_tile+1)*array_size, o_nij] =␣
 ↪out[oc_tile*array_size:(oc_tile+1)*array_size, o_nij] + \
                psum[ic_tile, oc_tile, :, int(o_nij/o_ni_dim)*a_pad_ni_dim +␣
 ↪o_nij%o_ni_dim + int(kij/ki_dim)*a_pad_ni_dim + kij%ki_dim, kij]
                ## 4th index = (int(o_nij/30)*32 + o_nij%30) + (int(kij/3)*32 +␣
 ↪kij%3)
```

[14]:
```
out_2D = torch.reshape(out, (out.size(0), o_ni_dim, -1))
difference = (out_2D - output_int[0,:,:,:])
print(difference.sum())
```

```
tensor(0.0004, device='cuda:0', grad_fn=<SumBackward0>)
```

[15]:
```
out.shape
out2 = torch.reshape(out, (len(ocg), -1))
out2.shape
#print(out)
```

[15]:
```
torch.Size([8, 16])
```

[ ]:
```
####### Store activation ####################
tile_id = 0
nij = 0 # just a random number
X = a_tile[tile_id,:,:]  # [tile_num, array row num, time_steps]

bit_precision = 4
file = open('activation_tile0.txt', 'w') #write to file
file.write('#time0row7[msb-lsb],time0row6[msb-lst],....,time0row0[msb-lst]#\n')
file.write('#time1row7[msb-lsb],time1row6[msb-lst],....,time1row0[msb-lst]#\n')
file.write('#.............#\n')

for i in range(X.size(1)):  # time step
```

```
    for j in range(X.size(0)): # row #
        X_bin = '{0:04b}'.format(round(X[7-j,i].item()))
        for k in range(bit_precision):
            #print(X_bin[k])
            file.write(X_bin[k])
        #file.write(' ')  # for visibility with blank between words, you can use
    file.write('\n')
file.close() #close file
```

[17]:
```
print(a_tile.shape)
print(X.shape, nij)
```

```
torch.Size([1, 8, 36])
torch.Size([8, 36]) 0
```

[29]:
```
####### store weight(kij) ###############
tile_id = 0
kij = 0
bit_precision = 4


for kij in range (9):
    file_name = f'weight_itile0_otile0_kij{kij}.txt'
    W = w_tile[tile_id,:,:,kij] # w_tile[tile_num, array col num, array row␣
 ↪num, kij]

    file = open(file_name, 'w')  #write to file
    file.write('#col0row7[msb-lsb],col0row6[msb-lst],....,col0row0[msb-lst]#\n')
    file.write('#col1row7[msb-lsb],col1row6[msb-lst],....,col1row0[msb-lst]#\n')
    file.write('#................#\n')

    for i in range(W.size(1)):  # time step
        for j in range(W.size(0)): # row #
            if (W[i,7-j].item() >= 0):
                W_bin = '{0:04b}'.format(round(W[i,7-j].item()))
            else:
                W_bin = '{0:04b}'.format(round(W[i,7-j].item())+16)
            for k in range(bit_precision):
                file.write(W_bin[k])
            #file.write(' ')  # for visibility with blank between words, you␣
 ↪can use
        file.write('\n')
    file.close() #close file
```

[ ]:

[25]:
```
### Store psum ###
ic_tile_id = 0
oc_tile_id = 0


kij = 0
nij = 0

# psum[len(ic_tileg), len(oc_tileg), array_size, len(p_nijg), len(kijg)]


bit_precision = 16
file = open('psum.txt', 'w') #write to file
file.write('#time0col7[msb-lsb],time0col6[msb-lst],....,time0col0[msb-lst]#\n')
file.write('#time1col7[msb-lsb],time1col6[msb-lst],....,time1col0[msb-lst]#\n')
file.write('#................#\n')

for kij in range(9):
    psum_tile = psum[ic_tile_id,oc_tile_id,:,nij:nij+36,kij]
    for i in range(psum_tile.size(1)):  # time step
        for j in range(psum_tile.size(0)): # row #
            if (psum_tile[7-j, i].item() >= 0):
                psum_tile_bin = '{0:016b}'.format(round(psum_tile[7-j,i].
 ↪item()))
            else:
                psum_tile_bin = '{0:016b}'.format(round(psum_tile[7-j,i].
 ↪item())+65536)
            #print(W_bin,' ', W[7-j,i].item(), j, i, '\n')
            for k in range(bit_precision):
                file.write(psum_tile_bin[k])
            #file.write(' ')  # for visibility with blank between words, you␣
 ↪can use
        file.write('\n')
file.close() #close file
```

[26]:
```
psum_tile.shape
print(psum_tile)
print(out)
```

```
tensor([[   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000,   98.0000,  154.0000,  161.0000,   77.0000,    0.0000,
            0.0000,   84.0000,  147.0000,   91.0000,   56.0000,    0.0000,
            0.0000,   91.0000,  147.0000,  -28.0000,  -63.0000,    0.0000,
            0.0000,  161.0000,  147.0000,  -14.0000,   -7.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
        [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000, -106.0000, -191.0000, -136.0000, -103.0000,    0.0000,
```

```
            0.0000, -108.0000, -123.0000,  -89.0000,  -54.0000,    0.0000,
            0.0000,  -50.0000,  -75.0000,  -20.0000,  -45.0000,    0.0000,
            0.0000,  -84.0000,  -85.0000,  -75.0000,  -67.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
          [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000, -136.0000, -230.0000, -153.0000, -119.0000,    0.0000,
            0.0000, -140.0000, -147.0000,  -91.0000,  -56.0000,    0.0000,
            0.0000,  -65.0000,  -99.0000,  -28.0000,  -63.0000,    0.0000,
            0.0000, -103.0000, -111.0000, -104.0000,  -91.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
          [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000,  112.0000,  224.0000,  147.0000,  119.0000,    0.0000,
            0.0000,  140.0000,  147.0000,   91.0000,   56.0000,    0.0000,
            0.0000,   35.0000,   63.0000,   28.0000,   63.0000,    0.0000,
            0.0000,   49.0000,   63.0000,   98.0000,   91.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
          [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000, -126.0000,  -56.0000,   21.0000,    7.0000,    0.0000,
            0.0000,  -84.0000,  -21.0000,   77.0000,   42.0000,    0.0000,
            0.0000, -105.0000, -147.0000,  -28.0000,  -63.0000,    0.0000,
            0.0000, -147.0000, -175.0000, -112.0000,  -77.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
          [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000, -112.0000, -238.0000, -161.0000, -119.0000,    0.0000,
            0.0000, -140.0000, -147.0000,  -91.0000,  -56.0000,    0.0000,
            0.0000, -105.0000, -147.0000,  -28.0000,  -63.0000,    0.0000,
            0.0000, -161.0000, -175.0000, -112.0000,  -91.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
          [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000,  112.0000,  238.0000,  161.0000,  119.0000,    0.0000,
            0.0000,  140.0000,  147.0000,   91.0000,   56.0000,    0.0000,
            0.0000,  105.0000,  147.0000,   28.0000,   63.0000,    0.0000,
            0.0000,  161.0000,  175.0000,  112.0000,   91.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000],
          [   0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000,
            0.0000,  -71.0000, -134.0000,  -92.0000,  -63.0000,    0.0000,
            0.0000,  -92.0000,  -99.0000,  -42.0000,  -27.0000,    0.0000,
            0.0000,  -51.0000,  -84.0000,   -8.0000,  -18.0000,    0.0000,
            0.0000,  -73.0000,  -86.0000,  -53.0000,  -47.0000,    0.0000,
            0.0000,    0.0000,    0.0000,    0.0000,    0.0000,    0.0000]],
         device='cuda:0', grad_fn=<SelectBackward0>)
tensor([[  385.0000,   161.0000,  -182.0000,  -329.0000,   468.9999,   357.0000,
            14.0000,   -70.0000,   510.9999,   238.0000,  -126.0000,    -7.0000,
           322.0000,    70.0000,  -294.0000,   -42.0000],
          [ -587.0000,  -907.0000,  -770.0000,  -422.0000,  -779.0000, -1176.0000,
           -1014.9999,  -516.9999,  -718.0000, -1022.0000,  -820.9999,  -441.0000,
            -429.0000,  -606.0000,  -524.0000,  -289.0000],
          [ -688.9999,  -940.9999,  -811.9999,  -427.0000,  -881.9999, -1169.0000,
```

```
         -1036.9999,  -516.9999,  -773.9999, -1068.9999,  -890.9999,  -439.0000,
          -569.9999,  -732.9999,  -610.0000,  -294.0000],
        [  510.9999,   804.9999,   783.9999,   413.0000,   566.9999,   706.9999,
           489.9999,    14.0000,   245.0000,   462.0000,   378.0000,    63.0000,
           154.0000,   182.0000,   490.0000,   280.0000],
        [  287.0000,   371.0000,   -14.0000,  -217.0000,     7.0000,   259.0000,
           182.0000,    56.0000,  -245.0000,    84.0000,   364.0000,   245.0000,
          -224.0000,  -126.0000,    98.0000,   140.0000],
        [  -35.0000,   -35.0000,    84.0000,   231.0000,    -7.0000,   -77.0000,
          -448.0000,  -238.0000,   231.0000,   -14.0000,  -489.9999,  -343.0000,
           224.0000,   252.0000,  -154.0000,  -140.0000],
        [  440.9999,   497.0000,   364.0000,    91.0000,   427.0000,   608.9999,
           336.0000,   -84.0000,   440.9999,   531.9999,   238.0000,  -119.0000,
           378.0000,   461.9999,   224.0000,    56.0000],
        [ -616.0000,  -683.9999,  -550.9999,  -287.0000,  -854.0000,  -895.9999,
          -656.0000,  -427.0000,  -724.0000,  -726.9999,  -473.0000,  -334.0000,
          -456.0000,  -515.0000,  -390.0000,  -181.0000]], device='cuda:0',
       grad_fn=<CopySlices>)
```

[27]:
```
bit_precision = 16
file = open('out.txt', 'w') #write to file
file.write('#time0col7[msb-lsb],time0col6[msb-lst],....,time0col0[msb-lst]#\n')
file.write('#time1col7[msb-lsb],time1col6[msb-lst],....,time1col0[msb-lst]#\n')
file.write('#................#\n')

out_relu = relu(out)
print(out_relu)
for i in range(out_relu.size(1)):  # time step
    for j in range(out_relu.size(0)): # row #
        #Out_bin = '{0:016b}'.format(round(out[7-j,i].item()))
        if (out_relu[7-j, i].item() >= 0):
            Out_bin = '{0:016b}'.format(round(out_relu[7-j,i].item()))
        else:
            Out_bin = '{0:016b}'.format(round(out_relu[7-j,i].item())+65536)
        for k in range(bit_precision):
            #print(X_bin[k])
            file.write(Out_bin[k])
       #file.write(' ')  # for visibility with blank between words, you can use
    file.write('\n')
file.close() #close file
```

```
tensor([[385.0000, 161.0000,   0.0000,   0.0000, 468.9999, 357.0000,  14.0000,
           0.0000, 510.9999, 238.0000,   0.0000,   0.0000, 322.0000,  70.0000,
           0.0000,   0.0000],
        [  0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,
           0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,
           0.0000,   0.0000],
        [  0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,
```

```
          0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,
          0.0000,   0.0000],
        [510.9999, 804.9999, 783.9999, 413.0000, 566.9999, 706.9999, 489.9999,
         14.0000, 245.0000, 462.0000, 378.0000,  63.0000, 154.0000, 182.0000,
        490.0000, 280.0000],
        [287.0000, 371.0000,   0.0000,   0.0000,   7.0000, 259.0000, 182.0000,
         56.0000,   0.0000,  84.0000, 364.0000, 245.0000,   0.0000,   0.0000,
         98.0000, 140.0000],
        [  0.0000,   0.0000,  84.0000, 231.0000,   0.0000,   0.0000,   0.0000,
          0.0000, 231.0000,   0.0000,   0.0000,   0.0000, 224.0000, 252.0000,
          0.0000,   0.0000],
        [440.9999, 497.0000, 364.0000,  91.0000, 427.0000, 608.9999, 336.0000,
          0.0000, 440.9999, 531.9999, 238.0000,   0.0000, 378.0000, 461.9999,
        224.0000,  56.0000],
        [  0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,
          0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,   0.0000,
          0.0000,   0.0000]], device='cuda:0', grad_fn=<ReluBackward0>)
```

[28]:
```
##### The following code is just for me to verify #####
##### Please Ignore #####
file = open('out.txt', 'r')
result = []

# Obtain the value in the last column
for line in file:
    line = line.strip()
    if not line or line.startswith("#"):
        continue
    parts = line.split()
    last_col = parts[-1]
    result.append(last_col)

print('The last column in out.txt is\n')
print(result)

def twos_complement_to_int(num_str, bits=16):
    num_str = num_str.zfill(bits)
    num = int(num_str, 2)
    if num >= 2**(bits-1):
        num -= 2**bits
    return num

print('\nConvert binary number into decimal number\n')
result_int = [twos_complement_to_int(x) for x in result]
print(result_int)
```

```
The last column in out.txt is
```

```
['00000000000000000000000011011100100000000000000000000000010001111100000001111111
11000000000000000000000000000000000000000000110000001', '00000000000000000000000111
11000100000000000000000000000010111001100000011001001010000000000000000000000000000
00000000000000010100001', '00000000000000000000000010110110000000000010101000000000
00000000000000001100010000000000000000000000000000000000000000000000000000000000000', '00
00000000000000000000000010110110000000011100111000000000000000000000011001110100
00000000000000000000000000000000000000000000000', '00000000000000000000001101010
11000000000000000000000000000001110000001000110111000000000000000000000000000000000
00000000111010101', '00000000000000000000010011000010000000000000000000000000100
00001100000010110000110000000000000000000000000000000000000000000101100101', '000000
00000000000000001010100000000000000000000000000010110110000000011110101000000000
00000000000000000000000000000000000001110', '00000000000000000000000000000000000
00000000000000000000001110000000000000011100000000000000000000000000000000000
00000000000000', '00000000000000000000001101110010000000111001110000000000000000
00000000011110101000000000000000000000000000000000000000111111111', '0000000000
00000000000100001010000000000000000000000001010100000000011100111000000000000
00000000000000000000000000011101110', '00000000000000000000011101110000000
00000000000000001011011000000001011110100000000000000000000000000000000000000000
0000000000', '00000000000000000000000000000000000000000000000000000001111010100
00000001111110000000000000000000000000000000000000000000000', '00000000000000
00000000101111010000000011100000000000000000000000000010011010000000000000000
00000000000000000000000101000010', '00000000000000000000001110011100000000011
11110000000000000000000000001011011000000000000000000000000000000000000000001
000110', '00000000000000000000011100000000000000000000000000011000100000000
01111010100000000000000000000000000000000000000000000000', '00000000000000000000
00000011100000000000000000000000000100011000000001000110000000000000000000000
00000000000000000000000000000']
```

Convert binary number into decimal number

```
[34939619674084932261620622492033, 39376396776433344422693838979233,
28839152704961287189282414329856, 72100420506625019495172380426 24,
33830425393720158957271028531669, 48249950975964887312367573991781,
26620662608150262775584493862926, 103302160877740877 4144,
34939898930654972819461533270527, 42149382459138284143396224172270,
18856302685109633587634933530624, 4519470030982372851712,
29948516229775556634441160786242, 36603715730896718083215092285510,
17747108405003130462915969089536, 44367771033814258888551277363200]
```

[ ]:

[ ]:

[ ]: